ЛЕКЦИЯ № 5 / 90 МИНУТ

# Дизайн-токены и техническая дизайн-система

От разрозненных CSS-значений к управляемой библиотеке интерфейса

**Аудитория** 4 курс, направление ИСИТ

**Продолжительность** 90 минут (одна пара)

**Предварительные знания** семантический HTML, Flexbox, Grid, адаптивная вёрстка

**Результат** понимание 3 уровней токенов, темизации и состояний компонентов

## Зачем эта тема

Страница с кнопками ещё не является дизайн-системой. Система появляется там, где решения о цвете, размере, состоянии и поведении принимаются по общим правилам, а не отдельно в каждом CSS-селекторе.

## 1. План занятия

| Время | Тема | Активность |
|---|---|---|
| 0–10 мин | Проблема несогласованного интерфейса | Разбор ситуации |
| 10–25 мин | Дизайн-токены и уровни абстракции | Примеры и вопросы |
| 25–40 мин | CSS Custom Properties и каскад | Демонстрация кода |
| 40–55 мин | Типографика, отступы, цвета, поверхности | Разбор системы |
| 55–70 мин | Состояния компонентов и две темы | Матрица и код |
| 70–82 мин | Доступность, структура проекта, диагностика | Короткое демонстрационное упражнение |
| 82–90 мин | Мини-проверка и переход к практике | Устные ответы |

## 2. Зачем нужны токены: инженерная проблема

В веб-приложении Help Desk уже готовы страницы заявок, формы и административная панель. Разные разработчики использовали три близких оттенка синего для основных кнопок, разные радиусы и хаотичные отступы. При смене фирменного цвета приходится искать значения по десяткам файлов. При внедрении тёмной темы возникают ошибки контраста и пропущенные компоненты.

**Суть задачи.** Техническая дизайн-система не заменяет дизайн-макет. Она переводит повторяемые проектные решения в именованные, проверяемые и переиспользуемые правила.

Дизайн-токен — это именованное представление повторяемого проектного решения: цвета, расстояния, радиуса, типографического размера или уровня поверхности. В CSS токен часто реализован через custom property, но сам термин относится к уровню проектирования.

## 3. Дизайн-токен и CSS-переменная — не одно и то же

| Признак | Дизайн-токен | Произвольная CSS Custom Property |
|---|---|---|
| Назначение | Фиксирует общее правило дизайна | Может хранить любое значение для конкретного селектора |
| Смысл имени | Роль понятна команде: --color-text | Имя может быть техническим: --offset-x |
| Область | Обычно используется в нескольких компонентах | Может жить только внутри одного элемента |
| Контроль | Есть шкала, схема именования и правила изменения | Общих правил может не быть |

Пример: `--card-animation-progress` — корректное CSS Custom Property, но не обязательно токен дизайн-системы. `--space-4` и `--color-text-muted` становятся токенами, когда включены в согласованную шкалу и используются системно.

## 4. Три уровня архитектуры токенов

| Уровень | Что описывает | Пример |
|---|---|---|
| Примитивный (primitive) | Само значение, без контекстной роли | --base-blue-700: #1853a4 |
| Семантический (semantic) | Назначение значения в интерфейсе | --color-primary: var(--base-blue-700) |
| Компонентный (component) | Роль в конкретном компоненте | --button-bg-primary: var(--color-primary) |

```
/* tokens.css */
:root {
  --base-blue-700: #1853a4;                 /* примитив */
  --color-primary: var(--base-blue-700);    /* семантика */
  --button-bg-primary: var(--color-primary);/* компонент */
}
/* components.css */
.button--primary { background: var(--button-bg-primary); }
```

Если нужен новый фирменный цвет, меняется примитив. Если меняется роль основного действия, обновляется семантический токен. Если только первичная кнопка должна выглядеть иначе, меняется компонентный токен. Необязательно заводить три уровня для каждого мелкого свойства: важна оправданная и понятная архитектура.

## 5. Как работают CSS Custom Properties

- Декларация: `--name: value;` в `:root`, компоненте или другом CSS-селекторе.
- Использование: `color: var(--color-text);` — подстановка значения происходит при вычислении стиля.
- Обычные свойства вида `--name` наследуются по каскаду; локальное переопределение действует для элемента и потомков.
- Запасное значение: `var(--border, #777)` помогает при отсутствующей переменной; оно не исправляет неверный тип уже подставленного значения.
- Важно: `var()` нельзя использовать вместо имени CSS-свойства или для значения условия `@media`.
- Объявление через `@property` (дополнительно) позволяет описать тип, начальное значение и наследование.

```
/* Локальное переопределение */
:root { --color-surface: #ffffff; }
.panel { background: var(--color-surface); }
.panel--inverted { --color-surface: #192536; }
/* Fallback на отсутствие токена */
.caption { color: var(--color-caption, #546376); }
```

**Тонкость отладки.** Если переменная определена, но содержит неподходящее значение (например, `--color-text: 16px` для `color`), браузер не обязан вернуться к предыдущему CSS-объявлению. В DevTools нужно смотреть вычисленное значение, а не только исходный файл.

## 6. Типографическая шкала: иерархия вместо случайных размеров

Типографические токены включают семейство, размер, насыщенность, межстрочный интервал и иногда межбуквенное расстояние. Шкала должна отражать смысл: служебная подпись → основной текст → подзаголовок → заголовок страницы.

| Токен | Значение | Назначение |
|---|---|---|
| --font-size-xs | 0.75rem (12 px) | Метки, дополнительные данные |
| --font-size-sm | 0.875rem (14 px) | Подписи полей, вспомогательный текст |
| --font-size-base | 1rem (16 px) | Основной текст и поля ввода |
| --font-size-lg | 1.25rem (20 px) | Небольшой заголовок |
| --font-size-xl | 1.5rem (24 px) | Заголовок секции |
| --font-size-display | clamp(2.15rem, 3.5vw, 3.5rem) | Основной заголовок в гибкой шкале |

```
h1 {
  font-size: var(--font-size-display);
  line-height: var(--line-height-tight);
}
body {
  font: var(--font-size-base)/var(--line-height-body)
        var(--font-family-body);
}
```

`rem` масштабирует размеры относительно корневого шрифта; `clamp(min, preferred, max)` ограничивает адаптивный размер. Проверяйте читаемость при увеличении текста и переносе длинных слов. Не нужно делать всё адаптивным через `vw`.

## 7. Система отступов, размеров и радиусов

Для учебной системы берём базовую единицу 4 px и ограниченную шкалу: 4, 8, 12, 16, 20, 24, 32, 48 px. Это не запрет на другие числа, а правило принятия решений. Повторяющиеся промежутки должны использовать одинаковые роли.

| Группа | Токены | Использование |
|---|---|---|
| Микроотступы | --space-1, --space-2 | Иконка + подпись, текст ошибки |
| Компоненты | --space-3, --space-4, --space-5 | Поля, кнопки, группа контролов |
| Блоки и секции | --space-6, --space-8, --space-12 | Карточки, панели, секции |

| Группа | Токены | Использование |
|---|---|---|
| Радиусы | --radius-sm, --radius-md, --radius-lg | Кнопка → карточка → крупная панель |
| Высота и плотность | --button-height, --control-height | Единая высота интерактивных элементов |

Геометрия интерфейса складывается из базовых токенов, но компоненты не обязаны совпадать до пикселя. Важнее предсказуемый ритм: одинаковые формы имеют одинаковую высоту, одинаковые панели — одинаковый внутренний отступ.

## 8. Цветовые роли, поверхности, границы и тени

| Роль | Что отвечает |
|---|---|
| --color-page | Фон всей страницы |
| --color-surface | Основная карточка / панель |
| --color-surface-subtle | Подложка, разделение уровней |
| --color-text, --color-text-muted | Основной и дополнительный текст |
| --color-border, --color-border-control | Декоративная граница и видимая граница поля |
| --color-primary, --color-on-primary | Основное действие и текст на нём |
| --color-success / error / info / warning | Семантика обратной связи |

Поверхность — не просто цвет. Уровень может задаваться сочетанием фона, границы, тени и расстояния. Тень не должна быть единственным способом отделить обязательный элемент управления. Для простых карточек достаточно светлой границы и минимальной тени.

**Проверка контраста (WCAG 2.2, уровень AA).** Обычный текст — не менее 4.5:1; крупный текст — не менее 3:1. Важные нетекстовые границы и визуальные индикаторы управления — обычно от 3:1 относительно смежных цветов. Отдельно проверяются светлая и тёмная темы.

## 9. Состояния: компонент — это набор правил поведения

| Состояние | Селектор / атрибут | Ожидаемое поведение |
|---|---|---|
| Default | основной класс | Читаемость, понятное назначение |
| Hover | :hover | Небольшая обратная связь при указателе |
| Active | :active | Ощущение нажатия |
| Keyboard focus | :focus-visible | Видимый контур для клавиатуры |
| Disabled | :disabled | Нет действия, явная недоступность |
| Invalid | [aria-invalid="true"] | Видимая ошибка + текстовое объяснение |

```
button:focus-visible,
input:focus-visible {
  outline: 3px solid var(--color-focus-ring);
  outline-offset: 3px;
}
.input[aria-invalid="true"] {
  border-color: var(--color-error);
}
```

Одна только смена цвета недостаточна для ошибки. У поля должны быть подпись, признак ошибочности и текстовое пояснение, связанное через `aria-describedby`. Нельзя удалять стандартный outline, не обеспечив сопоставимого фокуса.

## 10. Светлая, тёмная и системная темы

Разделяйте базовые значения и их роли. Компонент обращается к `--color-surface`, а не к HEX-фону напрямую: при смене темы переопределяются семантические значения, а компонент остаётся тем же.

| Режим | Как определяется | Приоритет |
|---|---|---|
| Светлая | Атрибут data-theme="light" | Ручной выбор |
| Тёмная | Атрибут data-theme="dark" | Ручной выбор |
| Системная | Атрибута data-theme нет; действует prefers-color-scheme | Системное предпочтение |

```
/* В светлой теме значения объявлены в :root */
:root { --color-page: #f6f8fc; color-scheme: light; }
@media (prefers-color-scheme: dark) {
  :root:not([data-theme]) {
    --color-page: #101923;
    color-scheme: dark;
  }
}
:root[data-theme="dark"] {
  --color-page: #101923;
  color-scheme: dark;
}
```

`prefers-color-scheme` сообщает о настройке ОС или браузера; свойство `color-scheme` подсказывает браузеру, как оформить встроенные элементы (поля, полосы прокрутки). Одно не заменяет другое. Приоритет ручного выбора надо реализовать явно.

```
function applyTheme(choice) {
  const root = document.documentElement;
  if (choice === 'auto') root.removeAttribute('data-theme');
  else root.dataset.theme = choice;
}
/* выбор можно сохранить в localStorage */
```

## 11. Уважение пользовательских настроек и доступность

- У формы должны быть корректные `label`, а у ошибки — читаемое сообщение, не только красная граница.
- Все кнопки и селекторы должны быть доступны через Tab и работать с клавиатуры.
- Состояние `disabled` задаётся HTML-атрибутом, а не только снижением прозрачности.
- Анимация и скролл должны учитывать `@media (prefers-reduced-motion: reduce)`.
- Проверка включается на ширинах 320, 375, 768, 1024, 1440 px и при длинном тексте.
- Контраст оценивается по конкретной паре «текст/фон», а не по красоте набора цветов.

## 12. Организация небольшого UI Kit

```
project/
  index.html              # семантическая страница-компонентов
  css/tokens.css          # значения и темы
  css/components.css      # внешний вид и состояния
  js/theme.js             # выбор темы и сохранение
  README.md               # как запустить и проверить
```

Порядок импортов важен: сначала `tokens.css`, затем стили компонентов. Не смешивайте статические значения из макета с правилами семантики. Новые роли добавляйте только при наличии понятного случая использования.

## 13. Типичные ошибки

| Антипаттерн | Почему плохо | Как исправить |
|---|---|---|
| HEX во всех классах | Тема расходится в деталях | Семантические токены |
| --blue1, --blue2, --blue3 | Название не выражает назначение | --color-primary, --color-text-muted |
| Привязка темы к отдельным кнопкам | Усложнение селекторов и сопровождения | Переопределить роли на корне |
| Только :hover | Не видно фокуса с клавиатуры | :focus-visible |
| Слишком много значений | Шкала теряет смысл | Ограниченный набор и критерии расширения |
| Отдельная копия CSS для dark | Дублирование и расхождение поведения | Общий CSS компонентов + темы в tokens.css |

## 14. Демонстрация и вопросы к группе

1. Поменяйте `--color-primary` и найдите три элемента UI Kit, у которых изменился цвет без правки их классов.
2. Откройте DevTools → Computed. Объясните, откуда взялся `background-color` первой кнопки.
3. Покажите разницу между `--base-blue-700`, `--color-primary` и `--button-bg-primary`.
4. Объясните, почему ручная светлая тема не должна превращаться в тёмную при настройке ОС.

**5.** Предложите действие при недостаточном контрасте подсказки поля в тёмной теме.

## Контрольные вопросы

- Может ли произвольная CSS-переменная считаться дизайн-токеном? При каких условиях?
- Почему на уровне компонента стоит использовать семантический токен, а не HEX?
- Как работают наследование и fallback в `var()`?
- Какие UI-состояния необходимо продемонстрировать на технической странице?
- Какая настройка имеет приоритет: явный выбор темы или системная?
- Почему переключение темы недостаточно тестировать только визуально?

## Источники и дополнительное чтение


- MDN — Using CSS custom properties: https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Cascading_variables/Using_custom_properties
- MDN — prefers-color-scheme: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-color-scheme
- MDN — color-scheme: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/color-scheme
- MDN — prefers-reduced-motion: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-reduced-motion
- W3C WCAG 2.2 — Contrast (Minimum): https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum
- W3C WCAG 2.2 — Non-text Contrast: https://www.w3.org/WAI/WCAG22/Understanding/non-text-contrast


После лекции студент должен уметь объяснить причинно-следственную связь «примитив → роль → компонент», показать её в DevTools и сформулировать план проверки тёмной темы.